Project Report

Machine Learning

Computer Vision Challenge – Sports Analytics

Team members:

Julia Drygalska r0912428

Pierina Lopez r0913865

Thomas Van Sande r0655289

TABLE OF CONTENTS

INTRODUCTION

In this project, we worked on a system for detecting tennis players and the ball in match videos and visualizing their positions on a mini-court map.

The goal was to create a tool that can automatically process tennis match videos, detect what is happening on the court, and present it clearly for match analysis.

For our project, we integrated several technologies:

- YOLOv8 for real-time object detection of players, the ball, and court areas
- Roboflow for dataset preparation and annotations
- OpenCV for video processing, frame extraction, and drawing visual elements such as mini court
- Streamlit for creating an interactive interface where we could upload images and instantly visualize detection results

By combining these components, we created a tool capable of analyzing tennis matches visually. It can detect key objects, track player and ball positions and display it dynamically on a mini-map.

Data and Preprocessing

We used 4 short tennis match videos recorded from normal camera angles. Since the full videos are large and training requires a lot of images, we decided to extract frames from the videos using OpenCV. To reduce redundancy and save time, we took one frame every 0.5 seconds (2 FPS).

```
video= "src/raw/video1.mp4"
out_dir = "src/videos/frames/video_one"
os.makedirs(out_dir, exist_ok=True)

cap = cv2.VideoCapture(video)
fps = cap.get(cv2.CAP_PROP_FPS)
i = 0
while True:
    ret, frame = cap.read()
    if not ret: break
    if i % int(fps//2) == 0:  # 2 FPS
        cv2.imwrite(f"{out_dir}/frame_{i}.jpg", frame)
    i += 1
cap.release()
```

Our code automatically creates a folder for the frames, reads the video, and saves one image every half second. We repeated this process 2 more times for other videos.

The resulting frames we used later as the base for our Roboflow annotations of different court areas.

ANNOTATION

We used Roboflow to annotate the court areas, not the players or the ball, because we used pretrained YOLO models for these elements. The goal of this dataset was to help the model learn to recognize and separate different zones of a tennis court.

The annotated classes were:

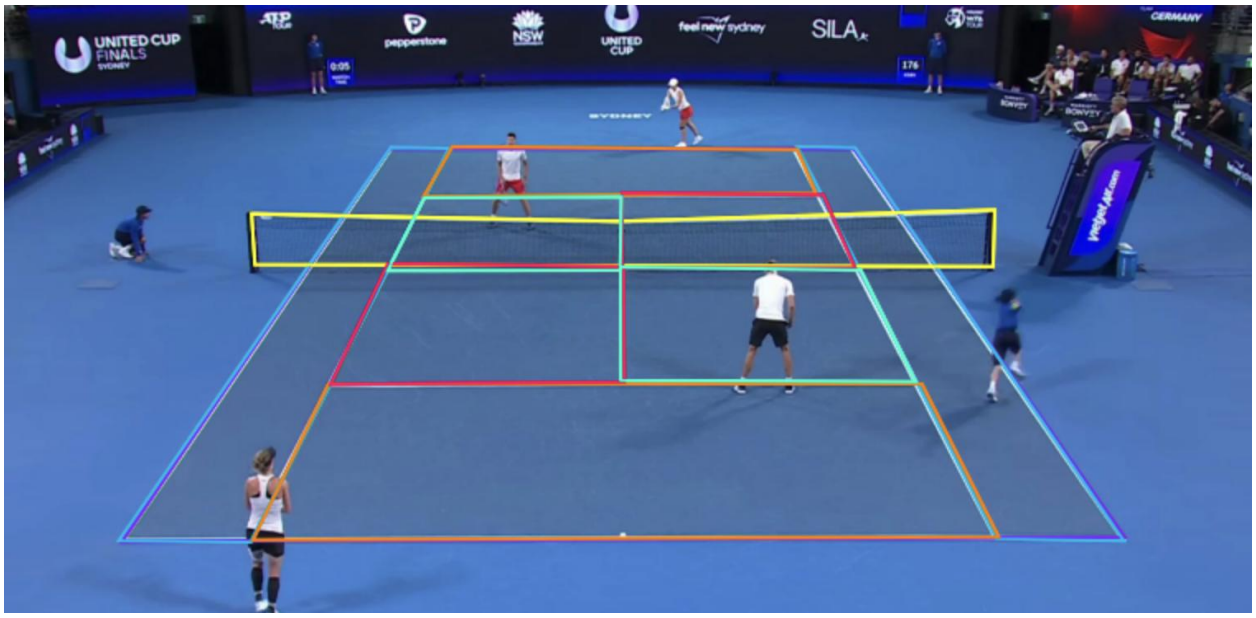
- Court
- Backcourt
- Doubles alley
- Left service box
- Right service box
- Net

In total, our dataset contains 82 images, split into :

- Training: 60
- Validation: 15
- Test: 7

Since annotations were created with Roboflow, a YOLOv8-compatible dataset was automatically generated. It included a data.yaml file with class names and paths, as well as folders for images and labels.

The image below shows one of our annotated frames. Each color represents a different part of the court.



To download the dataset and prepare it for YOLOv8 training, we used the Roboflow API:

```
rf = Roboflow(api_key="s928UxAwmihcQRRAd7tA")
project = rf.workspace("machine-learning-6flte").project("tennis-dpx0e")
version = project.version(2)
dataset = version.download("yolov8")
```

This code automatically connects to the project, downloads the correct version of our annotated dataset, and organizes it in a structure ready for YOLOv8 training.

Model Training

For the training phase, we used the YOLOv8n (nano) model, a lightweight architecture optimized for speed and efficiency, a perfect solution for our relatively small dataset.

We started from pretrained weights (yolov8n.pt) trained on the COCO dataset and fine-tuned the model on our custom tennis court dataset. The training was executed for 15 epoch with image size of 640x640 using the following code:

```python
model = YOLO("yolov8n.pt")
model.train(
    data="tennis-2/data.yaml",
    epochs=15,
    imgsz=640
)
```

The YOLOv8 framework automatically logged the training progress, saving metrics, visualizations, and model weights inside the runs/detect/train directory.

After training, we analyzed the performance metrics from the results.csv file to see how accuracy evolved over time.

```python
# Load the training results
results = pd.read_csv("runs/detect/train15/results.csv")

# Display average metrics
print(results[["metrics/precision(B)", "metrics/recall(B)", "metrics/mAP50(B)", "metrics/mAP50-95(B)"]].mean())

# Plot mAP50-95 over time
plt.plot(results["epoch"], results["metrics/mAP50-95(B)"])
plt.title("mAP50-95 per Epoch")
plt.xlabel("Epoch")
plt.ylabel("mAP50-95")
plt.show()
```

The final validation results showed that the model successfully learned to detect and distinguish between all six court areas. The average precision (mAP@0.5) across all classes reached 0.723, while the stricter mAP@0.5–0.95 averaged 0.681.

The best-performing classes were Court (mAP50 = 0.995) and Doubles Alley (mAP50 = 0.995), showing near-perfect detection accuracy.

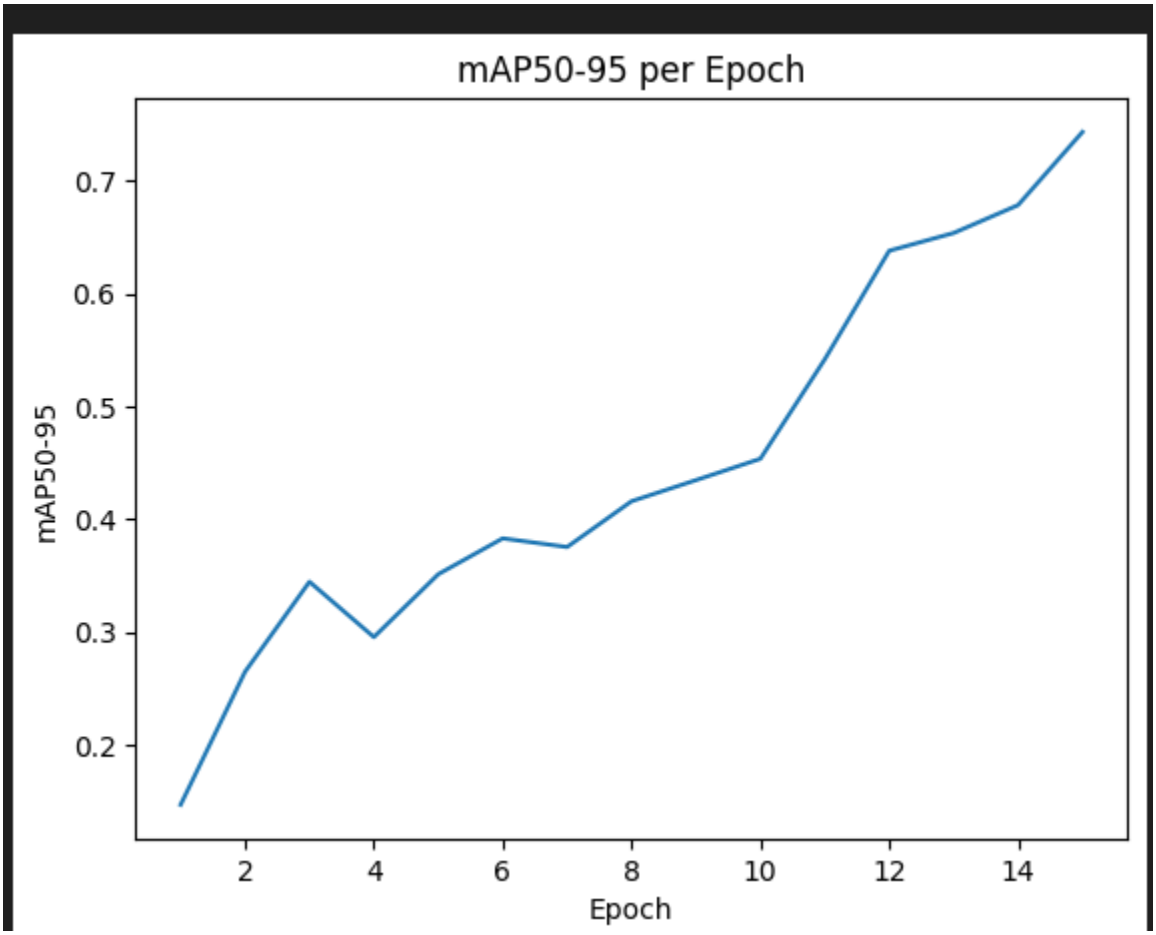
The Left Service Box and Right Service Box achieved slightly lower scores (both around 0.79 mAP50), while Net remained challenging for the model to detect consistently.

Overall, these results are strong considering the size of our dataset and a short 15-epoch training period.

The model demonstrates solid performance for larger and more distinctive areas of the court but would likely improve from more annotated samples and longer training to improve recall and accuracy for smaller parts of court like service boxes or net.

```
Validating C:\Users\thoma\Downloads\CV_Challenge_Team_1_Tennis 1\CV_Challenge_Team_1_Tennis\runs\detect\train17\weights\best.pt...
Ultralytics 8.3.214  Python-3.13.8 torch-2.8.0+cpu CPU (11th Gen Intel Core i7-11700 @ 2.50GHz)
Model summary (fused): 72 layers, 3,006,818 parameters, 0 gradients, 8.1 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────────── 1/1 0.6it/s 1.8s
                   all         15        150      0.931      0.444      0.723      0.681
             backcourt         15         30      0.672      0.533      0.768      0.723
                 court         15         15      0.946          1      0.995      0.995
         doubles alley         15         30      0.967          1      0.995      0.992
      left service box         15         30          1          0       0.79      0.681
                   net         15         15          1          0          0          0
     right service box         15         30          1      0.133       0.79      0.694
```

We also plotted the mAP50- 95 values across all epochs which showed a gradual improvement in accuracy over time.



After training was complete, we used best-performing model saved as best.pt for inference on a test video:

```python
model = YOLO("runs/detect/train15/weights/best.pt")
results = model.predict(source="input/input_video.mp4", show=False, save=True)
```

The model processed 384 frames of a tennis match video.

Each frame typically contained detections of 1 backcourt, 1 court, and 1 doubles alley, demonstrating consistent recognition of the main court zones.

The average inference time per frame ranged between 25–60 ms, allowing for near real-time detection.

The results confirmed that the model can effectively identify key parts of the tennis court.

External Pretrained Models

In order to obtain reliable, real-time detections of the player and the tennis ball and convert them into a top-down view of the court for analytics (heatmaps, zones, possession, trails).

Models Used:

- YOLOv8n(yolov8n.pt): general-purpose object detector; we use the built-in person class for player detection.

  Why: small, fast, and accurate enough for "person" without extra training.

- Tennis Ball Detector(best_ball.pt): pretrained YOLOv8 specialized for small, fast balls.

  Why: Tennis balls are tiny and often blurred; a specialist model improves recall/precision versus a generic detector.

```
model_players = YOLO("yolov8n.pt")
model_ball = YOLO("ball_model/best_ball.pt")
```

Rationale for This Setup

- Transfer learning efficiency: Reuse strong "person" priors in YOLOv8n → no need to label players.
- Small-object specialization: Generic models struggle with small, fast objects; the ball model addresses that.

How the Pipeline Works

1. Load once
   - model_players = YOLO("yolov8n.pt")
   - model_ball = YOLO("ball_model/best_ball.pt")
2. Per-frame inference
   - results_p = model_players(frame)

     Ultralytics runs NMS internally; we iterate results_p[0].boxes.

- results_b = model_ball(frame)

  Same, then scan results_b[0].boxes.

3. Player extraction (person class)
   - For each player box: cls_id == 0 (person) and conf > 0.50.
   - From the bbox (x1, y1, x2, y2) we compute a footpoint proxy at the bottom-center:

     cx = (x1+x2)/2, cy = y2, then add a small grounding bias cy += 0.10*h (with h=y2-y1) to better anchor feet.

   - We also draw the bbox and a dot at the bottom-center for visualization.
4. Ball extraction (keep-one-ball rule)
   - Among all ball boxes, we keep the highest-confidence detection with conf > 0.40.
   - From the bbox we compute the center (bx, by) as the ball point for this frame.
5. Temporal smoothing (ball)
   - We apply an exponential moving average to (bx, by) using BALL_EMA_ALPHA to reduce jitter and bridge brief dropouts.

```
# -------- Detect Players --------
results_p = model_players(frame)
feet_points_img = []
for box in results_p[0].boxes:
    cls_id = int(box.cls[0])
    conf   = float(box.conf[0])
    if cls_id == 0 and conf > 0.5:  # 'person'
        x1b, y1b, x2b, y2b = box.xyxy[0]
        x1b, y1b, x2b, y2b = float(x1b), float(y1b), float(x2b), float(y2b)
        cx = (x1b + x2b) / 2.0
        cy = y2b
        h  = y2b - y1b
        cy_adj = min(frame_h - 1, cy + 0.10 * h)  # feet grounding bias

        feet_points_img.append([cx, cy_adj])
        # Draw on original frame
        cv2.rectangle(frame, (int(x1b), int(y1b)), (int(x2b), int(y2b)), (0, 255, 0), 2)
        cv2.circle(frame, (int(cx), int(cy)), 4, (255, 0, 0), -1)
```

```python
# -------- Detect Ball --------
results_b = model_ball(frame)
ball_mini_pt = None
best_conf = 0.0
for box in results_b[0].boxes:
    conf = float(box.conf[0])
    if conf > 0.40 and conf > best_conf:
        x1, y1, x2, y2 = box.xyxy[0]
        x1, y1, x2, y2 = float(x1), float(y1), float(x2), float(y2)
        bx = (x1 + x2) / 2.0
        by = (y1 + y2) / 2.0
        pt = np.array([[[bx, by]]], dtype=np.float32)
        mini_ball = cv2.perspectiveTransform(pt, H)[0][0]
        mini_ball[0] = np.clip(mini_ball[0], 0, court_w - 1)
        mini_ball[1] = np.clip(mini_ball[1], 0, court_h - 1)
        ball_mini_pt = (float(mini_ball[0]), float(mini_ball[1]))
        best_conf = conf
```

Key Implementation Choices

- Model pairing: Generic YOLOv8n is ideal for people; a specialized YOLOv8 handles the small, fast ball.
- Class-specific thresholds: person > 0.50, ball > 0.40 were empirically tuned for our footage.
- Keep-one-ball: Prevents duplicate ball boxes and simplifies downstream logic.
- Footpoint heuristic (players): Using bottom-center of the bbox better approximates on-court location than bbox center.
- Light smoothing: EMA on the ball position improves stability without heavy tracking machinery.

What This Enables

- Reliable player & ball overlays in the broadcast view.
- Ball trails and basic tempo/rally segmentation (presence, speed in pixels/sec).
- Player activity traces (via IDs) and simple proximity heuristics in image space.
- Automatic highlight cues: fast rallies, long exchanges, or peak ball speeds.

We combine a fast, general person detector (YOLOv8n) with a specialist tennis-ball model. The code filters classes, applies tuned thresholds, keeps a single ball per frame, and smooths the ball signal-producing clean, stable detections that power the later analytics and visuals.

HOMOGRAPHY AND MINI-COURT DRAWING

Because the videos were filmed from a camera angle, we used a homography transformation to project the detected elements (players and ball) onto a flat, top-down 2D “mini-court”.

To start we have to create our own custom map. To do that we defined the width, height etc. We choose all these things ourselves. The following code is the drawing of said mini-court.

```
canvas_width, canvas_height = 300, 550
court_width, court_height = 250, 500
padding = 25

mini_court = np.zeros((canvas_height, canvas_width, 3), dtype=np.uint8)
mini_court[:] = (200, 150, 100)

top_left = (padding, padding)
top_right = (padding + court_width, padding)
bottom_right = (padding + court_width, padding + court_height)
bottom_left = (padding, padding + court_height)
corners = np.array([top_left, top_right, bottom_right, bottom_left], np.int32)
cv2.polylines(mini_court, [corners], isClosed=True, color=(255, 255, 255), thickness=3)

cv2.line(mini_court,
         (padding, padding + court_height//2),
         (padding + court_width, padding + court_height//2),
         (255,255,255), 2)

singles_margin = 15
cv2.line(mini_court,
         (padding + singles_margin, padding),
         (padding + singles_margin, padding + court_height),
         (255,255,255), 2)
cv2.line(mini_court,
         (padding + court_width - singles_margin, padding),
         (padding + court_width - singles_margin, padding + court_height),
         (255,255,255), 2)
```

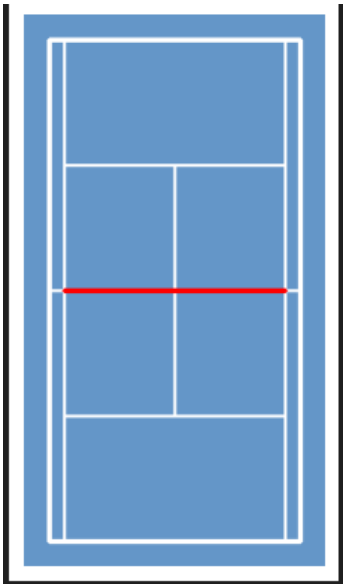
```
service_line_offset = court_height//4
cv2.line(mini_court,
         (padding + singles_margin, padding + service_line_offset),
         (padding + court_width - singles_margin, padding + service_line_offset),
         (255,255,255), 2)
cv2.line(mini_court,
         (padding + singles_margin, padding + court_height - service_line_offset),
         (padding + court_width - singles_margin, padding + court_height - service_line_offset)
         (255,255,255), 2)

cv2.line(mini_court,
         (padding + court_width//2, padding + service_line_offset),
         (padding + court_width//2, padding + court_height - service_line_offset),
         (255,255,255), 2)

net_y = padding + court_height//2
cv2.line(mini_court,
         (padding + singles_margin, net_y),
         (padding + court_width - singles_margin, net_y),
         (0,0,255), 3)


plt.imshow(cv2.cvtColor(mini_court, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

As you can see we created all the lines of a real tennis court and at the end to check how it looks we quickly make it show up in the output. The resulting output looks like this:



Then to make it easy for ourselves we quickly save this image as a png image in our folder structure so we can easily refer to it for later use.

```
output_path = "mini_court.png"
cv2.imwrite(output_path, mini_court)
print(f"Mini court saved to {output_path}")
```

Now that we have our mini-court the issue is finding a way to convert the real court on the video onto the mini-court so that the players and ball can be accurately represented on the mini-court. With that we can later use this to add some cool analysis to our project and clearly show where the players and ball are at any given moment.

To do this we use homography. We define the source points, these are the pixel coordinates of the corners of the tennis court on the video that are calculated by the trained model.

```
src_pts = np.float32([
    [466.7, 516.5],
    [2395.3, 516.5],
    [2395.3, 1316.4],
    [466.7, 1316.4]
])
```

Then we define our destination points, these are the coordinates of the four corners of the minimap.

```
dst_pts = np.float32([
    [padding, padding],
    [padding + court_width, padding],
    [padding + court_width, padding + court_height],
    [padding, padding + court_height]
])
```

With these points we can now apply homography. What this will do is calculate a matrix that allows each point in the real court to accurately be depicted on the mini-court.

```
H, _ = cv2.findHomography(src_pts, dst_pts)
```

TENNIS ANALYTICS – METRICS & HEATMAPS ADD-ON

We extended the baseline YOLO inference pipeline into a mini analytics suite for tennis.

The system now:

1. Thacks players and the ball on a homography-aligned mini-court
2. Generates both cumulative and live (video) heatmaps
3. Estimates ball speed, player distance run, and zone occupancy
4. Computes possession proxy (closest player to the ball with smoothing and hysteresis)

Player and Ball Heatmaps

Cumulative heatmaps:

Instead of incrementing a single pixel per frame, we add a filled disk ("splat") around each observation:

- splat _haet(array, x, y, radius, gain) = paints a small disk to make tracks visible
- Saved using a color map wth Gaussian blur and gamma correction to emphasize mid-intensities

Live, fading heatmap

We maintain trail heatmaps that decay every frame (half-life ~2.5 s). These are blended over the mini-court in the corner of the video.

Outputs:

- Heatmap_ball.png
- heatmap_player_<ID>.png (one per tracked player)
- output_video_with_players.mp4 (now includes live heatmap)

Player Tracking with Stable IDs

We keep simple nearest-neighbor matching in mini-court space to provide stable player IDs frame-to-frame:

- Gate: DIST_GATE_PX (≈ 60 px or ~22% of mini-court width).
- We compute and accumulate distance run (meters) by converting mini-court pixel deltas to meters using court scale.

Possession Metric (Closest-to-ball)

Possession is approximated as the player closest to the smoothed ball point on the court plane. To reduce flicker:

- Ball smoothing: exponential moving average (EMA) with BALL_EMA_ALPHA≈0.55–0.65.
- Hysteresis radii (meters): acquire if distance ≤ POSS_ACQUIRE_M (≈3.0–3.6 m), hold unless distance ≥ POSS_RELEASE_M (≈4.0–4.6 m).

- Persistence: keep possession for POSS_PERSIST_S (~0.30–0.50 s) if the ball briefly disappears.

We accumulate possession time per player and show the active possessor in the HUD

Speed and Distance Metrics

- Ball speed: per-frame court-plane distance / converted to km/h, then smoothed with an EMA (SPEED_ALPHA≈0.3).
- We ignore outliers where per-frame court distance exceeds a sanity threshold (MAX_FRAME_DIST_M≈8–10 m) to avoid spikes from missed frames.
- Players: total distance run and average speed (distance/time observed).

Zone Occupancy

We split the mini-court into interpretable zones:

- Side: Far vs. Near (relative to camera).
- Depth: Backcourt (baseline ↔ service line) vs. Forecourt (service line ↔ net).
- Wing: Ad vs. Deuce (left vs. right halves).

We accumulate seconds per zone for each player and list top zones in the summary.

```
===== MATCH ANALYTICS SUMMARY =====
Duration: 6.4 s (0.1 min)
Ball speed avg: 271.3 km/h | max: 4396.9 km/h

-- Player P1 --
  Time on court: 3.6 s
  Distance run:  9.3 m
  Avg speed:     9.2 km/h
  Possession:    0.5 s (7.5%)
  Top zones:     Far-Backcourt-Deuce:2.6s, Far-Forecourt-Deuce:1.1s

-- Player P2 --
  Time on court: 6.4 s
  Distance run:  6.8 m
  Avg speed:     3.8 km/h
  Possession:    0.0 s (0.0%)
  Top zones:     Near-Backcourt-Deuce:6.4s
```

VISUALIZATION AND TRACKING

As the last element to the project we created a streamlit web application to have a nice way to display our results on a webpage. To accomplish things we made it so that when we are doing a prediction on the input video, we also store every frame as an image. From that image folder we then chose 5 images that showed when the tracking was successful and when sometimes the ball would disappear.

```
frame_names = [f[0] for f in frames_info]
caption_dict = dict(frames_info)

# --- Initialize session state index ---
if "idx" not in st.session_state:
    st.session_state.idx = 0

# --- Navigation buttons ---
col1, col2, col3 = st.columns([1, 3, 1])
with col1:
    if st.button("⬅️ Prev"):
        st.session_state.idx = (st.session_state.idx - 1) % len(frame_names)
with col3:
    if st.button("➡️ Next"):
        st.session_state.idx = (st.session_state.idx + 1) % len(frame_names)

# --- Load current frame ---
selected_frame_name = frame_names[st.session_state.idx]
caption = caption_dict[selected_frame_name]
```

We started with loading the images from the folder within the streamlit folder and then followed it up with creating an id so we know which image is displaying. We then created easy to use navigation buttons and then loaded the current frame.

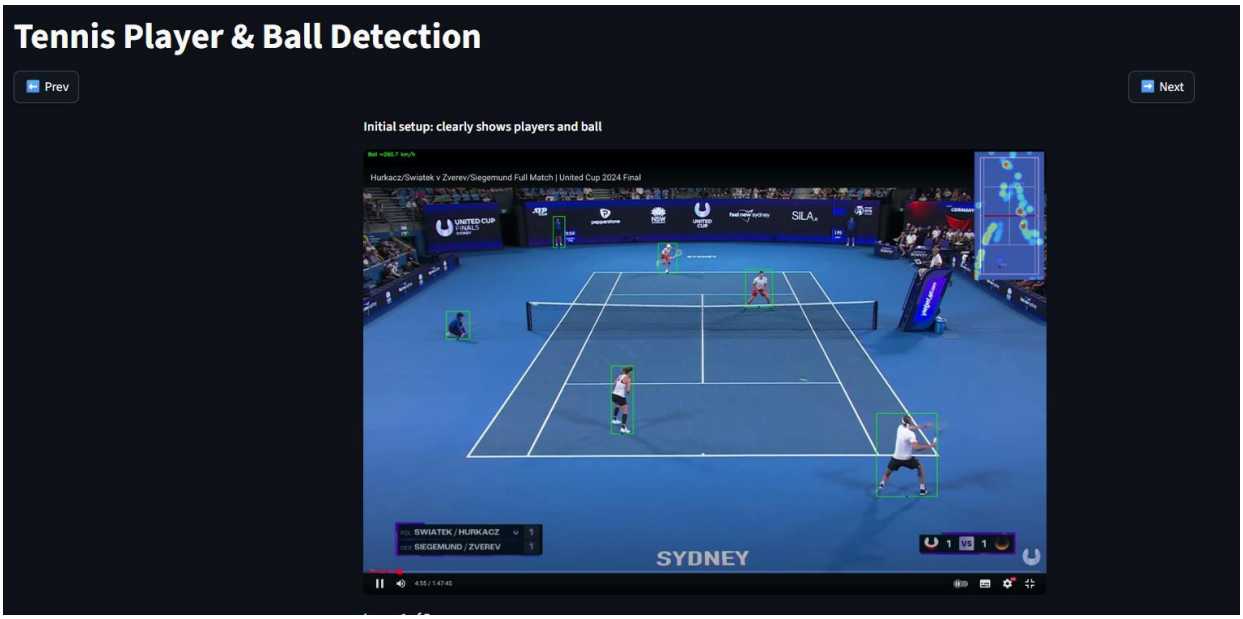
```
if not os.path.exists(selected_frame_name):
    st.error(f"Image not found: {selected_frame_name}")
else:
    # --- Center the image, caption, and frame index ---
    cols = st.columns([1, 2, 1])  # left, middle, right
    with cols[1]:  # middle column
        # Caption above image
        st.markdown(f"**{caption}**")

        # Load and resize image
        image = Image.open(selected_frame_name)
        max_width = 1000  # adjust to your screen
        width_percent = max_width / float(image.size[0])
        new_height = int(float(image.size[1]) * width_percent)
        image_resized = image.resize((max_width, new_height), Image.Resampling.LANCZOS)

        # Display resized image
        st.image(image_resized)

        # Frame index below image
        st.markdown(f"**Image {st.session_state.idx + 1} of {len(frame_names)}**")
```

To finalize we display the images and do so by centering them on the page. The result looks like the following:



You can see that the players are always recognized in their detection. The ball is a bit trickier. You can see in 2 images that whenever the ball is kind of hidden in front or behind a player, the model has a hard time detecting the ball, and the result is that the ball detection will disappear for a while on the mini-court until it is a little bit more visible again, and then it'll reappear. Also, a persistent problem is that the player position on the mini-court is always a little bit too high up. To try to solve this, we tried using the coordinates of the bottom of the bounding box so it coincided with the feet of the player, and that made it a little bit better, but still not quite perfect.

Conclusion

So to summarize, we used 3 models to train the detection of players, ball, and court zones on a tennis court. We used those models to predict on an untrained tennis video to predict object detection. We then also projected those detections on a mini-court and added heatmaps to it as well. We also printed some interesting analytics at the end like distance covered and speed of the ball. To have a nice way to present our project, we created a Streamlit web application to show the results.

The project was a nice way to get a feeling for how these object detection models work. In the beginning, we needed a lot of help from things like video tutorials to help us understand what the different phases of the process were. As expected, there were some issues at the beginning, especially with the players being correctly tracked on the mini-court. In the end, we weren't able to perfectly position them, but we're still happy with the project we delivered and have gained a better practical and conceptual understanding of object detection using computer vision.

AI Use & Reflection

Tool: OpenAI ChatGPT (GPT-5 Thinking).

Used for: planning/clarifications, drafting report & slides, and light code ideas (e.g., class thresholds, keep-one-ball, EMA, footpoint heuristic).

Not used for: data collection/labeling, training/evaluation, or final code decisions—those were done and verified by us.

Validation: all AI suggestions were tested on our footage and adjusted; our repo remained the single source of truth.